

Внешний курс 3 этап

Архитектура компьютеров и операционные системы

Венис Дмитрий

2026-05-03

Содержание I

- 1 Информация
- 2 Цель работы
- 3 Задание
- 4 Выполнение работы
- 5 Выводы

Раздел 1

Информация

- Венис Дмитрий



- Венис Дмитрий
- студент НКАбд-01-25



- Венис Дмитрий
- студент НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы



- Венис Дмитрий
- студент НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы
- 1032253513@rudn.ru



- Венис Дмитрий
- студент НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы
- 1032253513@rudn.ru
- <https://github.com/VenisDmitry>



Раздел 2

Цель работы

Цель работы

Целью работы является прохождение третьего этапа внешнего курса «Введение в Linux» и закрепление практических навыков работы с редактором `vim`, написанием `bash`-скриптов, использованием условий, циклов и функций, применением команд `find`, `grep`, `sed`, `gnuplot`, а также управлением правами доступа и файловыми утилитами Linux.

Раздел 3

Задание

Задание

Необходимо пройти задания третьего этапа внешнего курса и подготовить отчёт по всем выполненным активностям. Для каждого тестового вопроса и практического задания требуется привести:

- скриншот с формулировкой задания;

Задание

Необходимо пройти задания третьего этапа внешнего курса и подготовить отчёт по всем выполненным активностям. Для каждого тестового вопроса и практического задания требуется привести:

- скриншот с формулировкой задания;
- скриншот, подтверждающий успешное прохождение;

Задание

Необходимо пройти задания третьего этапа внешнего курса и подготовить отчёт по всем выполненным активностям. Для каждого тестового вопроса и практического задания требуется привести:

- скриншот с формулировкой задания;
- скриншот, подтверждающий успешное прохождение;
- краткое пояснение по выбору ответа или выполнению задания.

Раздел 4

Выполнение работы

Текстовый редактор vim

Указал, что для выхода из редактора vim сразу после открытия файла нужно нажать :, затем q, затем Enter (Рисунок 1).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim?

Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Здорово, всё верно.

- ☐ "Esc"
- ☒ " : ", затем "q", затем "Enter"
- ☐ "q", затем "Enter"
- ☐ " : ", затем "q"
- ☐ "Ctrl", затем "x"

Верно решил **47 171** учащийся

Из всех попыток **67%** верных

Следующий шаг

Решить снова



Определил корректные наборы нажатий клавиш, позволяющие преобразовать строку one two three four five в строку three four four four five (Рисунок 3).

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже **наборов нажатий клавиш** выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Правильно.

Верно решили **33 967** учащихся
Из всех попыток **16%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ d2wwywPp
- ☒ xxxxxxxxwywPp
- ☒ d2w\$bifour four <Esc>
- ☒ d2wwifour four <Esc>
- ☐ d2wwywpp
- ☒ ddithree four four four five<Esc>

Самостоятельно разобрался с визуальным режимом vim и отметил правильные утверждения о его работе (Рисунок 5).

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Верно решили **34 235** учащихся
Из всех попыток **29%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☐ Режим выделения открывается при помощи команды :visual
- ☒ В режиме выделения можно использовать команды d (удалить) и y (скопировать)
- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"

Скрипты на bash: основы

Указал, что при перемещении по истории команд стрелками вверх и вниз после запуска нескольких вложенных оболочек будут показываться только команды из текущей оболочки, то есть из набора С (Рисунок 6).

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Хорошая работа.

Верно решили **44 085** учащихся
Из всех попыток **63%** верных

- ☐ Только из набора В
- ☐ Из наборов В и С
- ☒ Только из набора С
- ☐ Из наборов А и С
- ☐ Только из набора А

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: ...

Определил, что после выполнения приведённого скрипта файл `file1.txt` не будет существовать, поэтому корректным ответом является вариант «Никак» (Рисунок 7).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

- ☒ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/file1.txt`
- ☐ `/home/bi/Desktop/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`

Отправить

Верно решили **43 649** учащихся
Из всех попыток **73%** верных

Ваши решения Вы получили: **1 балл**

Отметил допустимые имена переменных в bash: `_variable`, `VARiable`, `variable123` (Рисунок 8).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✓ Правильно.

Верно решили **39 540** учащихся
Из всех попыток **25%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ var-i-able
- ☐ var@i-able
- ☒ _variable
- ☐ var i-able
- ☒ VARi-able
- ☐ 123variable
- ☒ variable123

Следующий шаг

Решить снова

Написал скрипт, принимающий два аргумента командной строки и выводящий их в формате Arguments are: \$1=... \$2=... (Рисунок 9).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```



Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Так точно!

Верно решил **36 601** учащихся
Из всех попыток **40%** верных

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 echo "Arguments are: $1=$1 $2=$2"
2
3
4
5
6
```



Скрипты на bash: ветвления и циклы

В задании по условным выражениям отметил только те варианты, при которых команда `echo "True"` будет выполняться всегда, независимо от параметров запуска скрипта и значений переменных (Рисунок 10).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты *могут* изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Верно.

Верно решили **33 718** учащихся
Из всех попыток **16%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ -z ""
☒ -e \$0

Определил, что при запуске фрагмента скрипта с `var=3`, а затем с `var=5`, на экран сначала выводится `four`, а потом снова `four` (Рисунок 11).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент `bash`-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Прекрасный ответ.

Верно решили 37 107 учащихся
Из всех попыток 62% верных

- ☒ Сначала `four`, потом `four`
- ☐ Сначала `two`, потом `four`
- ☐ Сначала `four`, потом `one`
- ☐ Сначала `two`, потом `one`

Написал скрипт, который по количеству студентов выводит одну из строк: No students, 1 student, 2 students, 3 students, 4 students или A lot of students (Рисунок 12).

Напишите скрипт на `bash`, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверять вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно**!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin` → `stdout`

✓ Абсолютно точно.

Верно решили **34 567** учащихся
Из всех попыток **41%** верных

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -eq 0 ]]; then
2     echo "No students"
3 elif [[ $1 -eq 1 ]]; then
4     echo "1 student"
5 elif [[ $1 -eq 2 ]]; then
6     echo "2 students"
7 elif [[ $1 -eq 3 ]]; then
8     echo "3 students"
9 elif [[ $1 -eq 4 ]]; then
10    echo "4 students"
11 else
12    echo "A lot of students"
```

Указал, что в данном фрагменте цикла слово `start` выводится 5 раз, а слово `finish` — 4 раза (Рисунок 13).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: loops1.sh, loops2.sh.

Посмотрите на фрагмент `bash`-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово `"start"`, а сколько раз слово `"finish"`?

Выберите один вариант из списка

☒ Хорошая работа.

Верно решили **36 404** учащихся
Из всех попыток **46%** верных

- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☐ 3 раза "start" и ни разу "finish"
- ☐ 3 раза "start" и 2 раза "finish"

Написал интерактивный скрипт, который запрашивает имя и возраст пользователя, определяет возрастную группу (child, youth, adult) и завершает работу по пустому имени или возрасту 0 (Рисунок 14).

Напишите скрипт на bash, который будет определять к какому возрастной группе попадает пользователь. При запуске скрипт должен вывести сообщение "enter your name" и ждать от пользователя ввода имени (используйте read, чтобы прочитать его). Когда имя введено, то скрипт должен написать "enter your age" и ждать ввода возраста (опять же read). Когда возраст введен, скрипт пишет на экран "Hello, your group is <группа>". Так <группа> определяется на основе возраста по следующим правилам:

- младше либо равно 16 "child";
- от 17 до 25 (включительно) "youth";
- старше 25 "adult".

После этого скрипт опять выводит сообщение "enter your name" и всё начинается по новой (бесконечный цикл). Если в какой-то момент работы скрипта будет введено пустое имя или возраст 0, то скрипт должен написать на экран Bye и закончить свою работу (выйти из цикла).

Примеры корректной работы скрипта:

MP1

```
./script.sh
enter your name:
Egor
enter your age:
25
Egor, your group is child
enter your name:
Elena
enter your age:
8
Bye
```

MP2

```
./script.sh
enter your name:
Elena Petrenko
enter your age:
25
Elena Petrenko, your group is youth
enter your name:
Bye
```

Подсказка: в случае проблем с решением задания, обратитесь пожалуйста на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Проверить, молодец!

Всего решений 32 360 учащихся
Из всех решений 28% верные

Теперь вам доступны [форумы решений](#), где вы можете сравнить свое решение с другими или спросить совета.

На решение отведено 30 минут. Экспериментируйте, так как вы можете получить дополнительные 5 баллов за решение, отличившись на его вопросы, или сравнить своё решение с другими на [форуме решений](#).

```
1 while true
2 do
3     echo "enter your name:"
4     read name
5
6     if [[ -z "$name" ]]; then
7         echo "bye"
8         break
9     fi
10
11     echo "enter your age:"
12     read age
13
14     if [[ $age -eq 0 ]]; then
15         echo "bye"
16         break
17     fi
```

Скрипты на bash: разное

Отметил все инструкции, которые увеличивают значение переменной a на значение переменной b (Рисунок 15).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b` ? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

✔ Все правильно.

Верно решили 32 582 учащихся
Из всех попыток 20% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ✔ ☒ `let "a=$a+$b"`
- ☐ `let "a+=b"`
- ☐ `let a = a + b`
- ✔ ☒ `let a=a+b`
- ✔ ☒ `let a=$a+$b`

Определил, что после перехода в каталог `/home/bi/` команда `echo "pwd"` выведет на экран текст `pwd`, а не текущий путь (Рисунок 16).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: programs.sh.

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Так точно!

Верно решили 35 059 учащихся
Из всех попыток 51% верных

- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☒ `/home/bi`
- ☐ ``pwd``
- ☐ `/home/bi/Documents`
- ☐ `pwd`

Следующий шаг

Решить снова

Указал верные способы проверить код возврата внешней программы, если она пишет что-то в stdout (Рисунок 17).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполнятся, если программа закончилась с кодом 0). Однако это **не всегда правда!** Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, **используется** именно **косая кавычка** (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Верно решили **31 986** учащихся
Из всех попыток **22%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `if `program > some_file.txt``
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☐ Сначала `var=`program``, затем `if [[ $var -eq 0 ]]`
- ☐ `if [[ `program` -eq 0 ]]`

Определил результат десяти вызовов функции counter: строка, выводимая командой echo, имеет вид counters are and 110 (Рисунок 18).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
  local let "c1+=${1}"
  let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Отличное решение!

Верно решили **29 682** учащихся

Из всех попыток **28%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Написал рекурсивный скрипт для вычисления наибольшего общего делителя двух чисел по алгоритму Евклида (Рисунок 19).

Напишите скрипт на bash, который будет искать наибольший общий делитель ([НОД](#), greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждать ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "**GCD is «посчитанное значение»**", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "**bye**" и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД – он равен **M** (или **N**), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M больше N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем (**M-N**), а в качестве второго **N**. Если же наоборот, **M меньше N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым (**N-M**).

Пример корректной работы скрипта:

```
./script.sh
18 15
GCD is 5
7 3
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Показка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

Верно решили 28 308 учащихся
Из всех попыток 41% верных

✓ Так точно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 gcd() {
2     local m=$1
3     local n=$2
4
5     if [[ $n -eq $n ]]; then
6         echo "$n"
7     elif [[ $n -gt $n ]]; then
8         gcd $((n - m)) "$n"
9     else
10        gcd "$n" $((n - m))
11    fi
12 }
13
14 while read m n
15 do
16     if [[ -z "$n" && -z "$n" ]]; then
17         echo "bye"
18         break
```

Написал калькулятор на bash, который поддерживает операции +, -, *, /, %, **, завершает работу по команде exit и выводит error при некорректной команде (Рисунок 20).

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

- 1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
- 2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого перейти в режим ожидания новой команды.
- 3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./calculator.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
30 + 1
2 ** 38
exit
```

то на экране будет:

```
33
3024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/38
exit
```

то на экране будет:

```
error
```

т.к. вторая команда была **некорректной** (в ней всего один аргумент, т.к. нет пробелов между числами и операцией, а единственная доступная команда из одного аргумента это "exit").

Подсказки: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

Всё получилось!

Верно решено 26 557 учащихся
Из всех попыток 39% верные

Теперь вам доступны **Борисы решают**, где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 while [[ true ]]
3 do
4     read b1r1nch1 om1 i1k1nch1
5     if [[ $b1r1nch1 == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$b1r1nch1" =~ ^"([0-9]+) ([0-9]+) ([+|-|*|/|%|**])" ]]
10    then
11        echo "error"
12        break
13    else
14        case $om1 in
15            "+") let "result = $b1r1nch1 + $i1k1nch1";
16            "-" ) let "result = $b1r1nch1 - $i1k1nch1";
17            "/" ) let "result = $b1r1nch1 / $i1k1nch1";
18            "*" ) let "result = $b1r1nch1 * $i1k1nch1";
19            "%" ) let "result = $b1r1nch1 % $i1k1nch1";
20            "**") let "result = $b1r1nch1 ** $i1k1nch1";
```

Продвинутый поиск и редактирование

Указал, что команда `find /home/bi -iname "star*"` найдёт, а команда `find /home/bi -name "star*"` не найдёт файлы `STARS.txt` и `Star_Wars.avi` (Рисунок 21).

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдёт** команда `find /home/bi -iname "star*"`, но **НЕ найдёт** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Верно решили **30 748** учащихся
Из всех попыток **40%** верных

- ☒ STARS.txt
- ☐ star_trek OST.mp3
- ☐ Eddard_Stark_biography.txt
- ☐ stardust.mpeg
- ☒ Star_Wars.avi

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: **1 балл**

Рисунок 21: Сравнение `find -iname` и `find -name`

Отметил верные утверждения о различии опций `find -path` и `find -name` (Рисунок 22).

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

Верно решили **28 136** учащихся

Из всех попыток **25%** верных

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☒ В некоторых случаях `find` с `-name` найдет меньше файлов, чем `find` с таким же запросом, но с `-path`
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: **1 балл**

Рисунок 22: Понимание различий между `-path` и `-name` в `find`

Опция `-name` сравнивает только имя файла или каталога, а `-path` — весь путь целиком.

Определил, что команда `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"` найдёт все файлы, кроме `file3` (Рисунок 23).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
├── dir1  
│   ├── file1  
│   └── dir2  
│       ├── file2  
│       └── dir3  
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Абсолютно точно.

Верно решили **31 024** учащихся
Из всех попыток **42%** верных

- ☐ Все три файла
- ☐ Все кроме `file2`
- ☒ Все кроме `file3`
- ☐ Только `file2`
- ☐ Только `file3`

Отметил строки, которые выводит команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt` (Рисунок 25).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Верно решили 27 950 учащихся
Из всех попыток 23% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Mac OS X, Windows, Ubuntu
- ☐ Lubuntu is better than Windows
- ☐ Well, xubuntu is OK
- ☒ Linux is not always Ubuntu
- ☒ I prefer Kubuntu
- ☒ Lubuntu is better than Ubuntu

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: 2 балла

Рисунок 25: Регулярные выражения в `grep -E`

Строим графики в gnuplot

Указал, что при запуске `gnuplot` для сохранения построенных графиков после выхода из программы нужно использовать опцию `-p` (`--persist`) (Рисунок 27).

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Верно. Так держать!

Верно решили **28 374** учащихся
Из всех попыток **53%** верных

- ☒ `-p, --persist`
- ☐ Такой опции не существует
- ☐ `-s, --show-plots-after-exit`
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: **1 балл**

Рисунок 27: Опция `persist` при запуске `gnuplot`

Эта опция предотвращает автоматическое закрытие окон с графиками после завершения

Определил, что при командах `set key autotitle columnhead` и `plot 'data.csv' using 1:2` названием ряда данных станет первое значение из второго столбца, а на графике будет нарисовано 9 точек (Рисунок 28).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название у построенного ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

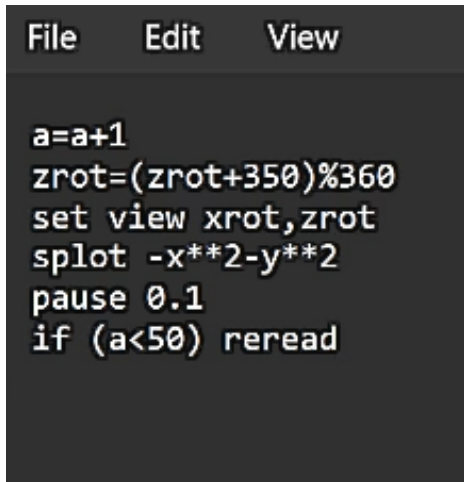
☒ Всё получилось!

Верно решили 27 337 учащихся
Из всех попыток 34% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название "data.csv using 1:2", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☐ Название "no name", нарисовано 10 точек
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)

Изменил файл `move.rot` так, чтобы график зеркально отразился относительно горизонтальной плоскости, начал вращаться в обратную сторону и делал это в два раза быстрее (Рисунок 29, Рисунок 30).



```
File Edit View

a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Рисунок 29: Изменённое содержимое файла `move.rot`

Отметил все команды, которые переводят права доступа файла `file.txt` из `r--r--r--` в `rw-rw-r--` (Рисунок 31).

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `r--r--r--`. Укажите **все** верные варианты ответа!

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнились все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Верно решили 24 856 учащихся
Из всех попыток 21% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
- ☐ `chmod rw-rw-r-- file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod 764 file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☐ `chmod u-wx file.txt; chmod g-w file.txt`

Следующий шаг

Решить снова

Указал команды, после которых пользователь user из группы group сможет создавать файлы в директории dir, созданной через sudo (Рисунок 32).

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r--r--` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно. Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `r` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволяют даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно! В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет посмотреть её содержимое (у него есть право `r`), но удалять и создавать файлы в ней не сможет (права `w` у него нет). Важно отметить, что директории в `Linux` это в каком-то смысле файлы. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `r` у директории дает возможность просматривать "записи", т.е. просматривать её состав. Право `w` у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый `sticky bit` (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

☒ Правильно, молодец!

Верно решили 22 695 учащихся
Из всех попыток 17% верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в комментариях, отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chmod o+w dir`
- ☒ `sudo chmod a+w dir`
- ☐ `chmod o+w dir`
- ☐ `sudo chmod o+x dir`
- ☒ `sudo chmod u+ac group:dir`

Отметил характеристики файла, которые можно посчитать при помощи команды `wc`: количество строк, слов и символов (Рисунок 33).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

 Отлично!

Верно решили **25 874** учащихся
Из всех попыток **23%** верных

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество слов
- ☐ Количество предложений
- ☒ Размер файла в байтах
- ☒ Количество строк
- ☒ Количество символов

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: **1 балл**

Рисунок 33: Использование команды `wc`

Вписал максимально короткую команду для создания трёх каталогов `dir1`, `dir2`, `dir3`:
`mkdir dir{1..3}` (Рисунок 35).

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст



Хорошая работа.

Верно решили **25 347** учащихся
Из всех попыток **46%** верных

```
mkdir dir{1..3}
```

Следующий шаг

Решить снова

[Ваши решения](#) Вы получили: **2 балла**

Рисунок 35: Создание нескольких директорий одной командой

В этом задании использовалось раскрытие фигурных скобок в оболочке `bash`, что позволяет

Раздел 5

Выводы

Выводы

В ходе выполнения третьего этапа внешнего курса были изучены и закреплены приёмы работы с редактором `vim`, написанием `bash`-скриптов, использованием позиционных параметров, условий, циклов, функций и рекурсии. Также были отработаны навыки поиска файлов и обработки текста командами `find`, `grep`, `sed`, основы построения графиков в `gnuplot`, а также управление правами доступа и использование утилит `chmod`, `ws`, `du` и `mkdir`. Полученные знания расширяют практические навыки работы в Linux и позволяют увереннее решать прикладные задачи автоматизации и обработки данных.